OFFLINE-FIRST COMMUNICATION PLATFORM FOR OUTDOOR AND EVENT MANAGEMENT

R26-IT-135

Project Proposal Report

Thanujana D O - IT22148704

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Thanujana D O	IT22148704	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature
Mr. Vishan Jayasingharachi	13/03/2026	

ABSTRACT

Effective communication within isolated areas or off-grid areas is a major challenge that has affected trekkers, event management teams, as well as emergency response teams that operate beyond mobile connectivity. The document below presents a description of the Short-Range Direct Communication Layer that is one of the integral parts of a collaborative initiative that is referred to as "Offline-First Communication Platform for Outdoor and Event Management" (Project ID: R26-IT-135).

The research gap that has been identified shows that existing offline communication tools such as GoTenna and Bridgefy are inadequate as far as offering instant delivery acknowledgments, minimal encryption, as well as the ability to engage in simultaneous private or group chats are concerned. The above is a major area that has been addressed by incorporating a layer that relies on a technology referred to as Bluetooth Low Energy that operates independently without the need for internet connectivity.

The proposed approach will be developed in an Android-based mobile application component that includes features like BLE-based peer-to-peer and group messaging, message delivery status, and AES-128-based data encryption. It will be developed following the Design Science Research (DSR) approach, in which iterative development and testing will be performed on actual devices. Testing will be conducted on actual Android-based smartphones in both indoor and outdoor environments to assess the rate of message delivery, delays, and Bluetooth connectivity range.

The expected outcome will be an Android-based BLE messaging component that ensures secure communication with an increased probability of over 90% in delivering messages within a range of 30-50 meters, with acknowledgement messages sent within two seconds, and data transmission encrypted using AES-128. It will have commercial viability in an expanding market related to outdoor activities, event management, and emergency situations, and can be sold either as an SDK or through licensing.

Keywords: Bluetooth Low Energy (BLE), Offline Messaging, Direct Communication, Message Delivery Confirmation, End-to-End Encryption, Mobile Application

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor Mr. Vishan Jayasinghe Arachchi and co-supervisor Ms. Sashini Hathurusinghe for their constant supervision, feedback, and encouragement during the development of this research proposal. I also extend my gratitude to the Department of Information Technology at SLIIT University for providing me with the necessary resources to complete this research successfully.

I extend my gratitude to my project group members (IT22168290, IT22544490, IT22894038) for their collaborative efforts that helped us complete the project framework as a whole, which is essential for this individual component as well.

Lastly, I extend my gratitude to my family and friends for their constant support during my academic pursuits.

Table of Contents

DECLARATION....2
ABSTRACT....3
ACKNOWLEDGEMENT....4
LIST OF TABLES....7
LIST OF FIGURES....7
LIST OF ABBREVIATIONS....7
LIST OF APPENDICES....8
1. INTRODUCTION....9
1.1 Background and Context....9
1.2 Literature Review....11
1.3 Research Gap....13
2. OBJECTIVE....15
2.1 Main Objective....15
2.2 Specific Objectives....15
3. METHODOLOGY....16
3.1 Assumptions....18
3.2 Technical Limitations....18
3.3 Research Approach....19
3.2 Technical Methods and Algorithms....19
3.3 Tools and Platforms....21
3.4 Data Collection and Ethical Considerations....23
3.5 Validation Strategy and Performance Metrics....24
4. HIGH-LEVEL SYSTEM ARCHITECTURE....24
4.1 Overall Group System Context....24
4.2 Component Internal Architecture....26
4.3 Data Flow....28
4.4 Scalability and Security Considerations....30
5. USER REQUIREMENTS....30
5.1 Stakeholder Identification....30
5.2 Functional Requirements....30
5.3 Non-Functional Requirements....31
6. COMMERCIALISATION PLAN....32

6.1 Target Market and Customer Persona 32
6.2 Value Proposition 33
6.3 Revenue Model 33
6.4 Cost Estimation 33
6.5 Competitive Advantage 34
6.6 Intellectual Property 34
7. BUDGET AND JUSTIFICATION 34
8. WORK BREAKDOWN STRUCTURE (WBS) 36
9. GANTT CHART 37
10. REFERENCES LIST 38
11. APPENDICES 39
Appendix A – BLE GATT Profile Specification 39
Appendix B – Risk Analysis 40

LIST OF TABLES

Table 1: Message Protocol Structure

Table 2: Tools and Platforms

Table 3: Validation Strategy and Performance Metrics

Table 4: Component Internal Architecture

Table 5: Functional Requirements

Table 6: Non-Functional Requirements

Table 7: Cost Estimation

Table 8: Budget and justification

LIST OF FIGURES

Figure 1: Research methodology flowchart

Figure 2- System Overall platform architecture

Figure 3 – Internal architecture

Figure 4 – Data flow diagram

Figure 5 – Work breakdown Structure (WBS)

Figure 6: Project Gantt Chart (March 2026 – July 2026).

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AES-128	Advanced Encryption Standard with 128-bit Key
API	Application Programming Interface
BLE	Bluetooth Low Energy
CACM	Context-Aware Communication Manager
DSR	Design Science Research

Abbreviation	Full Form
DTN	Delay-Tolerant Network
GATT	Generic Attribute Profile
MANET	Mobile Ad-hoc Network
P2P	Peer-to-Peer
SDK	Software Development Kit
TTL	Time-To-Live
UI	User Interface
UUID	Universally Unique Identifier
WBS	Work Breakdown Structure

LIST OF APPENDICES

Appendix A: BLE GATT Profile Specification.

Appendix B: Risk Analysis.

1. INTRODUCTION

1.1 Background and Context

The current society is highly dependent on the use of the internet to communicate through various messaging applications. In real-world cases such as hiking excursions, outdoor events, disaster response activities, and rural community events, the use of cellular or internet communication is often out of reach. According to the International Telecommunication Union, 2.6 billion people worldwide are yet to access the internet. In cases where the internet is accessible, it is often out of order during natural calamities or large-scale public events. Hikers, trekking groups, outdoor events, disaster response teams, among others, are some of the populations who are most affected by the inability to communicate a simple message.

Current consumer-grade applications like WhatsApp, Telegram, and Signal provide advanced features like read receipts and end-to-end encryption. However, they need a constant data connection and are completely unusable in an infrastructure-less environment. Partially offline applications like GoTenna Mesh and Bridgefy utilize Bluetooth or LoRa technology to provide internet-independent messaging. However, such applications have major limitations like the unavailability of real-time delivery confirmation in offline mesh networks, the inability to provide sufficient lightweight encryption on resource-constrained mobile devices, and a user interface centered around the use of hardware accessories instead of smartphones. Research on Delay Tolerant Networks (DTNs) and Mobile Ad-hoc Networks (MANETs) has been done to provide better routing mechanisms to deliver messages in an unconnected network. However, existing applications do not have the required features like delivery confirmation, encryption, and mobile support to be used in the real world outside.

In an effort to overcome these constraints, the proposed study is the construction of an Outdoor and Event Management Offline-First Communication Platform - a decentralized, infrastructure-independent Android communication platform that provides a reliable form of messaging without cellular dependencies, Wi-Fi or internet infrastructure dependencies. The platform is created to be used by hikers, event organizers of small to medium number attendees and emergency responders of remote or even disaster-stricken locations. It happens to run on standard Android smartphones, preventing any special hardware attachments, and can be used in places where a communication infrastructure is absent, further, it may exist in places where there is no communication infrastructure in place at all.

The platform is written in Android native Kotlin and follows a Hybrid Architecture with a Layered Architecture as a framework on how the system should be organized and an Event-Driven Architecture, as a reactive and asynchronous communication platform between components. This design has been specifically adopted to mirror the responsive and decentralized character of Bluetooth mesh networking, where the process of devices discovering each other, and messages being received, is event-driven and not command driven. This system helps to provide direct device-to-device messaging, over Bluetooth Low Energy, with a multi-hop forwarding via intermediate devices, opportunistic channel storage-carry-forward delivery where devices are disconnected, and also make smart routing choices based on real time information such as device proximity, battery status and message priority.

The particular researcher is the Short-Range BLE Direct Communication Layer - the basic communication building block of the system that enables direct encrypted real-time communication between physically close Android devices based on the Bluetooth Low Energy technology. This element fills a serious gap that has been listed under the current offline messaging solutions. Although other apps like Bridgefy do allow peer-to-peer Bluetooth communication, they do not have real-time confirmations of delivery in an offline scenario, fail to provide per-message encryption using dynamic initialization vectors and are not presented as native Android apps optimized for normal smartphone usage.

The component defines a special GATT service with special Message and Acknowledgement Characteristics that provide two-way communication between devices in both the Central and Peripheral mode - which has been offered since Android 5.0. Messages are encrypted with AES 128 using the CBC mode, the initialization message is generated at 128-bit with the Android secure random class, meaning that two different messages will never share the same initialization message. Connection time generates a session key in response to a nonce exchange communication over an exclusive Key Exchange Characteristic. Message status is monitored to four states (Pending, Sent, Dasher and Failed) with a 5 second acknowledgement delay with state changes visible to the user interface in real time with Kotlin State Flow. All messages are stored as persisted in a local Room database as encrypted messages so that no plaintext is stored on device at any point in time.

The component is structurally structured around the Repository pattern with all the hardware interactions of the View Model and UI layers being completely abstracted by a Message Repository class, making it possible to unit test hardware interactions without hardware involvement with mock data sources and also allow a clean separation of concern in keeping with current trends in Android development.

The study is in the Communication and Networking sub-topic of SST - Software Systems and Technologies under research division at SLIIT, and aligns with the SDG 9 (Industry, Innovation and Infrastructure), SDG 11 (Sustainable Cities and Communities), and SDG 3 (Good Health and Well-being) through building resilient, low-price, and accessible offline communication infrastructure in underserved settings.

1.2 Literature Review

An extensive review of academic and industry-related publications reveals an interesting perspective on current methodologies and their limitations in terms of offline direct communication.

1.2.1 Bluetooth Low Energy Mesh Networks for Offline Communication

Bluetooth Low Energy has emerged as a leading radio technology for short-range, infrastructure-independent communication due to its low power consumption and ubiquity in consumer mobile devices. Hernandez-Ramos et al. [1] examined BLE mesh network architectures and demonstrated that the managed flooding and friend/low-power node mechanisms defined in the Bluetooth Mesh Profile specification enable multi-hop message propagation in dense ad-hoc environments. The authors highlighted that BLE mesh is particularly suitable for event-scale deployments where devices form a self-organizing network without centralized coordination.

The Bluemergency system, proposed by Dias et al. [2], specifically targeted post-disaster communication by combining BLE mesh with Internet of Things (IoT) infrastructure. Their evaluation showed that BLE mesh nodes could relay emergency messages across multi-hop paths in situations where cellular towers were non-operational. A key contribution was demonstrating that lightweight encryption could be applied at the application layer without significantly degrading message throughput on resource-constrained devices. However, the system was limited in scalability beyond medium-density node deployments.

A comprehensive survey by de Morais Barroca Filho et al. [3] reviewed BLE mesh applications across healthcare, industrial IoT, and emergency response domains. The survey confirmed that delivery acknowledgements - analogous to the read receipts found in centralized messaging systems - remain largely unsupported in existing BLE mesh implementations, representing a significant usability gap for real-world deployment. The proposed platform directly addresses this gap by integrating delivery status indicators within its short-range direct communication layer.

1.2.2 Delay-Tolerant Networks and Opportunistic Routing

Delay-Tolerant Networking (DTN) was developed to address communication in environments characterized by intermittent connectivity, high latency, and frequent network partitioning [4]. The store-carry-forward (SCF) paradigm allows nodes to buffer messages locally and opportunistically deliver them upon contact with another node. Soares et al. [4] demonstrated that epidemic-based DTN routing achieves high delivery ratios in sparse mobile networks, though at the cost of increased buffer utilization. Time-To-Live (TTL) mechanisms were identified as essential for bounding message propagation and preventing network flooding - a design principle directly adopted in the proposed system's delay-tolerant transport module.

Trifunovic et al. [5] extended DTN research by proposing context-aware forwarding policies factoring in node mobility patterns, encounter history, and battery levels. Their work showed that integrating device context into routing decisions substantially improves delivery

probability while conserving energy. This finding directly informs the Context-Aware Communication Manager component of the proposed platform, which dynamically selects among BLE direct messaging, multi-hop routing, and delay-tolerant transport based on runtime device and network conditions.

1.2.3 Mobile Ad-hoc Networks and Decentralized Routing

MANETs provide a framework for fully decentralized, multi-hop communication where mobile devices act simultaneously as endpoints and relay nodes. Abolhasan et al. [6] conducted a comparative analysis of reactive, proactive, and hybrid MANET routing protocols, concluding that reactive protocols such as AODV (Ad hoc On-demand Distance Vector) offer favorable performance for moderate-size networks with dynamic topologies - a profile consistent with outdoor event and hiking trail deployments. The study further highlighted that end-to-end delivery acknowledgements are rarely implemented in existing MANET solutions due to overhead concerns in high-mobility scenarios.

Zhao et al. [7] proposed a lightweight MANET routing mechanism optimized for smartphone-based mesh networks, incorporating battery-awareness into the route selection algorithm. Their experimental results demonstrated that excluding low-battery nodes from relay paths extended overall network lifetime by up to 34%, supporting the integration of battery-level monitoring into the proposed system's context-aware manager. The absence of emergency message prioritization - noted as a limitation across reviewed MANET implementations - further motivates the dedicated emergency prioritization feature included in the proposed architecture.

1.2.4 Existing Offline Messaging Applications

Commercial offline messaging applications including Bridgefy and goTenna Mesh have demonstrated the practical viability of infrastructure-free communication at the consumer level. Avila-Campos et al. [8] evaluated Bridgefy's BLE-based mesh protocol in outdoor environments and found that effective communication range degraded significantly beyond 70 meters per hop in open terrain, with no built-in mechanism for long-distance message delivery. The study identified the absence of delivery confirmations and message prioritization as key usability deficiencies that limit adoption in safety-critical scenarios.

Aloi et al. [9] analysed infrastructure-independent communication in emergency management scenarios and proposed a hybrid architecture combining short-range BLE with optional fixed relay nodes deployed at strategic locations. Their findings demonstrated that even a small number of stationary relay devices substantially increased network coverage and message delivery rates in sparse user distributions. This aligns directly with the proposed platform's optional transmission repeater kit, designed to bridge coverage gaps along hiking trails or at event perimeters, thereby addressing a recognized limitation of purely device-to-device mesh solutions

.

1.2.5 Security and Privacy in Offline Mesh Networks

Security in decentralized mesh networks presents distinct challenges compared to server-mediated architectures, as messages traverse untrusted intermediate nodes. Graarud and Nergaard [10] reviewed lightweight encryption schemes suitable for BLE mesh environments and found that AES-128 in CCM mode - the standard mandated by the Bluetooth Mesh Profile -provides acceptable security guarantees with minimal computational overhead on modern smartphone hardware. The authors further noted that end-to-end encryption at the application layer is essential to protect message confidentiality even from relay nodes, a principle incorporated into the proposed system's encryption layer spanning all three communication modes.

1.3 Research Gap

Communication in infrastructure-deprived environments such as remote hiking trails, large outdoor events, and disaster-affected areas remains a persistent technological challenge. Conventional messaging platforms including WhatsApp and Telegram rely entirely on cellular or internet connectivity, rendering them non-functional where such infrastructure is absent. This review examines existing research across Bluetooth Low Energy (BLE) mesh networking, delay-tolerant networks (DTNs), mobile ad-hoc networks (MANETs), and offline messaging applications to establish the foundation for an offline-first communication platform.

BLE has emerged as a leading technology for short-range, infrastructure-independent communication due to its low power consumption and ubiquity in consumer smartphones. Hernandez-Ramos et al. [1] examined BLE mesh architectures and demonstrated that the managed flooding mechanisms defined in the Bluetooth Mesh Profile enable multi-hop message propagation in dense ad-hoc environments, making BLE mesh well suited for event-scale deployments. A comprehensive survey by de Morais Barroca Filho et al. [2] further confirmed that BLE mesh is widely adopted across healthcare and emergency response domains, though delivery acknowledgements remain a persistent unresolved limitation across existing implementations.

Delay-Tolerant Networking addresses communication in environments characterized by intermittent connectivity and frequent partitioning through the store-carry-forward (SCF) paradigm, where nodes buffer messages and deliver them opportunistically upon contact. Soares et al. [3] demonstrated that epidemic-based DTN routing achieves high delivery ratios in sparse mobile networks, identifying Time-To-Live mechanisms as essential for controlling message propagation and preventing network flooding. Trifunovic et al. [4] extended this work by proposing context-aware forwarding policies that incorporate node mobility and battery state into routing decisions, substantially improving delivery probability while conserving energy.

MANETs enable fully decentralized multi-hop communication where mobile devices act as both endpoints and relay nodes. Abolhasan et al. [5] conducted a comparative analysis of reactive, proactive, and hybrid MANET routing protocols, concluding that reactive protocols such as AODV perform favorably in dynamic topologies consistent with outdoor deployments. Zhao et al. [6] further advanced battery-aware route selection for smartphone-based MANETs, demonstrating that excluding low-battery nodes from relay paths extended network lifetime by up to 34%. However, both studies noted the consistent absence of end-to-end delivery acknowledgements and emergency message prioritization across existing MANET solutions.

At the application level, Avila-Campos et al. [7] evaluated Bridgefy's BLE mesh protocol and found that effective range degrades beyond 70 meters per hop with no long-distance fallback, while delivery confirmations and message prioritization are entirely absent. Aloi et al. [8] proposed a hybrid architecture combining BLE with optional fixed relay nodes, demonstrating that even a small number of stationary relays substantially improves coverage and delivery rates in sparse environments. Security considerations were addressed by Graarud and Nergaard [9], who confirmed that AES-128 encryption in CCM mode provides adequate confidentiality with minimal overhead, and that application-layer end-to-end encryption is essential to protect messages traversing untrusted relay nodes.

2. OBJECTIVE

2.1 Main Objective

Main Objective
The aim is to develop and implement a Short-Range Direct Communication Layer based on Bluetooth Low Energy technology that enables offline messaging with guaranteed reliability in one-on-one or multiple-group chats with real-time delivery proof and AES-128 encryption using only common Android devices without the need for an Internet connection or additional hardware.

2.2 Specific Objectives

1. To implement a BLE scanning mechanism using BluetoothLeScanner and ScanCallback that continuously discovers nearby devices and triggers the communication layer when a target device comes within range.
2. To develop a GATT-based message transmission system using BluetoothGatt that supports reliable one-to-one and group message delivery between devices over a direct BLE connection.
3. To design and implement a MessageRepository following the Repository pattern that fully abstracts the BLE hardware from the ViewModel, enabling the UI and application logic to remain independent of Bluetooth API details.
4. To integrate AES-128 lightweight encryption into the repository layer so that all message content is encrypted before transmission and decrypted upon receipt, ensuring user privacy in offline environments.
5. To implement real-time message delivery status indicators — Sent, Delivered, Pending, and Failed — using StateFlow that are observable by the UI layer and consistent with familiar messaging conventions.
6. To build a local message cache using Room database that stores sent and received messages for chat history and supports automatic retry of unacknowledged messages.
7. To implement an emergency alert mechanism that assigns a high-priority flag to emergency messages, overriding the normal message queue to ensure immediate transmission to all nearby devices.
8. To evaluate the performance of the BLE direct communication layer by measuring message delivery latency, connection reliability, and encryption overhead across a range of device distances and group sizes.

3. METHODOLOGY

Phase 1 - Requirement Analysis Identify the communication requirements for short-range offline messaging - device discovery range, message size limits, group size, encryption needs, and delivery confirmation behaviour - based on the use cases defined in the TAF (hikers, event organizers, emergency services).

Phase 2 - System Design Design the Repository pattern architecture for the BLE layer including the BLEViewModel, MessageRepository, BLE Scanner data source, GATT data source, and Room database cache. Define the StateFlow interfaces between the UI, ViewModel, and Repository layers.

Phase 3 - Implementation Develop the component in Android using Kotlin. Implement BluetoothLeScanner for device discovery, BluetoothGatt for data transfer, AES-128 encryption inside the Repository, Room database for message caching, and delivery status indicators via StateFlow. Integrate with Context aware Communication Manager event bus to receive Strategy A routing triggers.

Phase 4 - Testing Conduct unit testing by mocking the Repository layer to test ViewModel logic independently. Conduct integration testing using two or more physical Android devices in indoor and outdoor environments. Test edge cases including device going out of range mid-transmission, low battery, and emergency alert priority override.

Phase 5 - Evaluation Measure and record: message delivery latency at varying distances (1m, 5m, 10m, 15m), connection success rate, encryption and decryption overhead in milliseconds, and delivery confirmation accuracy. Compare results against existing offline messaging tools such as Bridgefy.

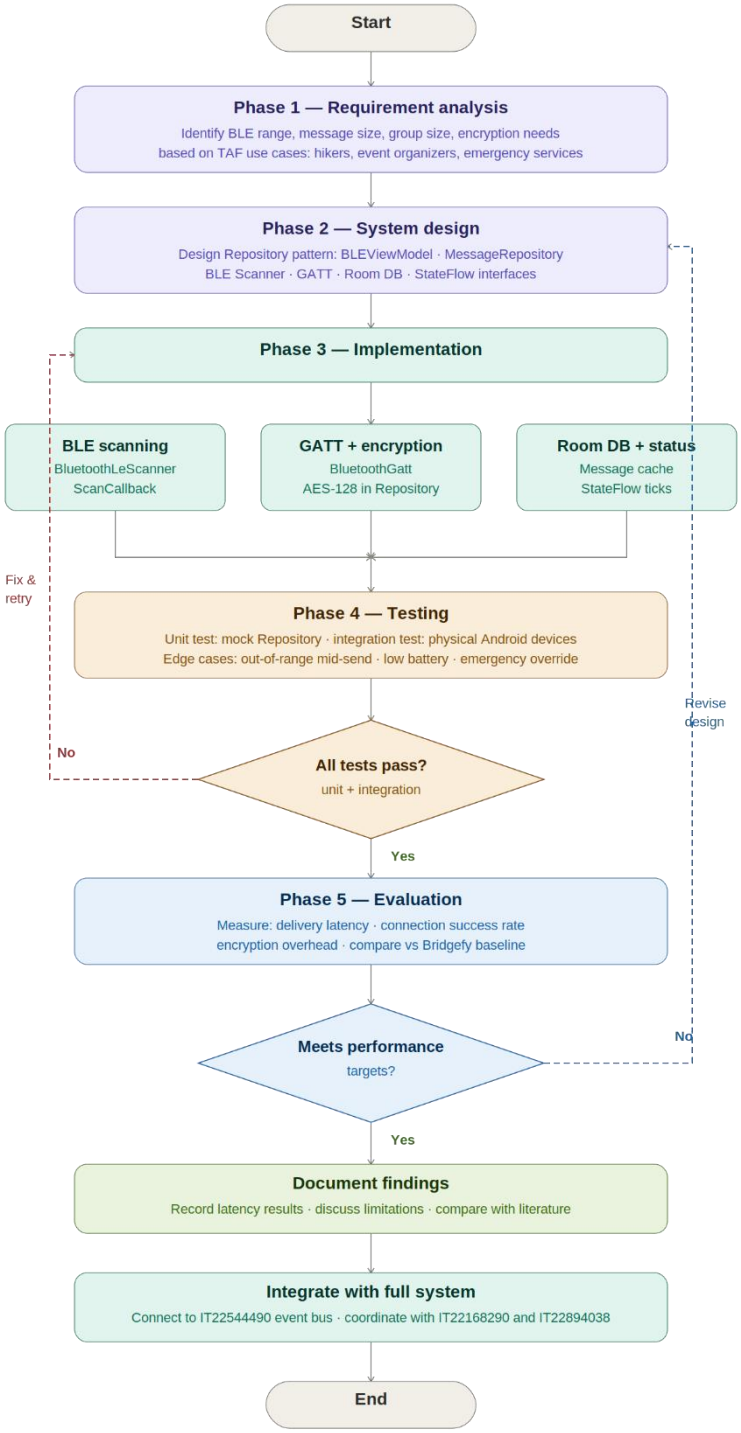


Figure 1 *– Research methodology flowchart*

3.1 Assumptions

1. All communicating devices are Android smartphones with BLE hardware support (Bluetooth 4.0 or above).
2. Devices are within the effective BLE transmission range of approximately 10–15 meters for reliable direct communication.
3. Users have granted the necessary Android runtime permissions — BLUETOOTH_SCAN, BLUETOOTH_CONNECT, and ACCESS_FINE_LOCATION.
4. The BLE radio on all test devices operates in a standard, non-interference environment during initial testing phases.
5. Message sizes are assumed to be small — text-based, under 512 bytes — consistent with typical chat message payloads.
6. The AES-128 shared key is pre-distributed among group members at session setup, and key management is outside the scope of this component.

Devices remain stationary or move slowly during direct BLE connection — fast movement causing frequent disconnections is treated as an edge case handled by the DTN component.

3.2 Technical Limitations

1. **BLE range** — effective range is approximately 10–15 meters in open space and significantly lower in environments with walls, crowds, or radio interference, limiting direct communication to close-proximity scenarios only.
2. **BLE connection throughput** — BLE is designed for low-power, low-bandwidth communication. Large file transfers (images, audio) are not feasible; the component is limited to text messages only.
3. **Android BLE API fragmentation** — BLE behavior varies across Android manufacturers and versions. Connection stability and scan intervals may differ between devices, affecting reliability in multi-device tests.
4. **Simultaneous connections** — Android imposes a practical limit of around 7 concurrent GATT connections per device, restricting group broadcast to small groups without additional mesh routing.
5. **AES-128 key distribution** — this component assumes a pre-shared key. A full key exchange protocol (such as Diffie-Hellman) is not implemented, which is a known security limitation for production use.
6. **Background scanning restrictions** — Android 8.0 and above impose aggressive background process restrictions that can throttle BLE scan intervals when the app is not in the foreground, potentially delaying device discovery.
7. **Battery consumption** — continuous BLE scanning has a measurable impact on device battery life, which may affect usability during extended outdoor activities. This is partially mitigated by the context manager reducing scan frequency when battery is critical.

3.3 Research Approach

The research will follow an iterative framework that consists of the following steps:

• Stage 1 – Identifying the Issue and Establishing Requirements: Evaluate the existing BLE documentation, the Android BLE API guide, and relevant literature; develop comprehensive functional and non-functional requirements.

• Stage 2 – Development of Design: Design the architecture of the BLE service/characteristic (GATT profile), the structure of the messaging protocol, the method of exchange of the encryption key, and the wireframes of the UI design.

• Stage 3 – Execution: Implement the Android module using the Flutter (Dart) language and the flutter_blue_plus BLE library, integrate the AES-128 encryption using the pointycastle Dart library, and design the chat interface.

• Stage 4 – Testing and Assessment: Conduct systematic experiments on actual devices and measure the outcomes against the established criteria of success.

3.2 Technical Methods and Algorithms

BLE Communication Protocol

This module employs a tailored GATT service featuring two attributes: one dedicated to message transmission (Message Characteristic – allows both notify and write) and another for confirming message receipt (Acknowledgement Characteristic – allows notify). Every device will function simultaneously in both Central and Peripheral capacities (a feature available since Android version 5.0 and later), eliminating the need for role switching during a peer-to-peer link.

Message Protocol Structure

The message data for BLE will be a binary packet formatted to comply with the BLE MTU specifications (standard value: 20 bytes; can be expanded up to 512 bytes through MTU extension):

Field	Size	Description
Message ID	4 bytes	Unique identifier for acknowledgement matching
Sender ID	6 bytes	Hashed device Bluetooth MAC address
Group Flag	1 byte	0x00 = direct, 0x01 = group broadcast

Timestamp	4 bytes	Unix timestamp (seconds)
Payload Length	2 bytes	Length of encrypted content
Encrypted Payload	Variable	AES-128 CBC encrypted message text

Encryption

This component employs AES-128 in CBC (Cipher Block Chaining) mode to encrypt all message payloads prior to BLE transmission, ensuring message confidentiality without reliance on internet-based security infrastructure.

Session Key Establishment

Upon establishing a GATT connection, each device generates a cryptographically random 128-bit nonce using Android's SecureRandom class. The two nonces are exchanged over a dedicated Key Exchange Characteristic, and a shared session key is derived by performing a bitwise XOR of both nonces. This ensures neither device transmits the session key directly over the air. While this approach is inspired by Diffie-Hellman and prevents passive eavesdropping, it is acknowledged that exchanging nonces in plaintext is a known cryptographic limitation. This is accepted given the short-range, low-threat offline environment and the undergraduate scope of the project. Full ECDH-based key exchange is identified as future work.

Message Encryption and IV Management

Each message payload is encrypted using the session key and AES-128 CBC mode. A unique 128-bit IV is generated via SecureRandom per message and prepended to the Encrypted Payload field in the binary packet. The receiver extracts the IV, decrypts the payload, and passes the plaintext to the application layer. If decryption fails, the message is silently discarded and no ACK is returned, causing the sender's delivery status to time out to Failed after 5 seconds.

Key Lifetime and Storage

Session keys are held in memory only and are never persisted to the Room database. Messages stored locally are retained in their encrypted form, ensuring that the database contains ciphertext rather than plaintext at all times.

Delivery Confirmation

When a decrypted and successfully received message is received, the device sends an acknowledgment packet containing the Message ID back over the Acknowledgement Characteristic. The status of the message is updated by the sender from “Sent” to “Delivered” after receiving the ACK. If the ACK is not received after 5 seconds, the status is updated to “Failed,” and the user is notified.

3.3 Tools and Platforms

Tool / Technology	Purpose	Justification
Kotlin	Primary programming language	Officially recommended by Google for Android; concise syntax, null safety, and native Coroutines support make it ideal for asynchronous BLE operations
Android Studio	IDE for development, debugging, and testing	Official Android development environment; provides built-in BLE debugging tools, Android profiler for performance measurement, and device emulator
Android BLE API (BluetoothLeScanner, BluetoothGatt)	BLE device discovery and GATT-based message transmission	Native Android API with full access to BLE hardware; supports simultaneous Central and Peripheral roles required by the dual-role peer-to-peer design
Jetpack Compose	UI layer for chat screen, group channel, and emergency alert	Modern declarative Android UI toolkit; natively observes StateFlow for real-time delivery status updates without manual UI refresh logic
ViewModel (androidx.lifecycle)	Holds message state and mediates between UI and Repository	Lifecycle-aware component that survives screen rotation; separates UI logic from BLE business logic as required by the Repository pattern

Tool / Technology	Purpose	Justification
StateFlow / SharedFlow (kotlinx.coroutines)	Reactive data streams for delivery status and event bus	Kotlin-native reactive streams; used to observe delivery status ticks and receive routing events from IT22544490 via the EDA event bus
Room Database (androidx.room)	Local message cache and undelivered message storage	Jetpack-supported SQLite abstraction; provides type-safe queries, coroutine-based async access, and persists encrypted message ciphertext locally — works fully offline
javax.crypto (AES-128 CBC)	Message payload encryption and decryption	Built-in Android cryptography library; no external dependency required; supports AES-128 CBC mode with per-message IV generation via SecureRandom
SecureRandom	Cryptographically random nonce and IV generation	Java security standard; guarantees unpredictable random values suitable for cryptographic use, unlike Random or Math.random()
JUnit 4 + Mockito	Unit testing of ViewModel and Repository logic	Standard Android unit testing stack; Mockito allows BluetoothLeScanner and BluetoothGatt to be mocked, enabling hardware-independent testing
Espresso	UI integration testing on physical devices	Official Android UI testing framework; validates delivery status tick updates on the chat screen under real BLE conditions
Android Profiler	Performance measurement during evaluation phase	Built into Android Studio; measures CPU, memory, and network activity to quantify BLE scan overhead and AES-128 encryption processing time

Tool / Technology	Purpose	Justification
Git / GitHub	Version control and team collaboration	Industry-standard source control; enables parallel development across the four team members with branch-based workflow and pull request reviews

3.4 Data Collection and Ethical Considerations

The information that will be used to examine the performance will be collected through a series of well-structured experiments involving at least three Android smartphones running Android 8.0 or later versions. The scenarios that will be used for testing are: (a) messaging between two devices at different distances (5m, 15m, 30m, 50m, 75m) while considering both indoor and outdoor environments; (b) group messaging involving three to five devices; and (c) measuring the cost of encryption/decryption operations.

The parameters that will be measured during the experiments are: percentage of messages delivered successfully, time taken for confirmation of messages at both ends (in milliseconds), range of messaging (in meters), and time taken for AES encryption/decryption operations (in milliseconds).

From an ethical perspective, the system will be collecting the BLE identifiers of the devices (hashed MAC address) and the encrypted payload of the messages. There will be no storage of personally identifiable information. The testing will be conducted on willing participants (the group members and their peers), who will be giving their verbal consent. Since no sensitive personal information will be collected, no Institutional Review Board approval is necessary. However, the set guidelines of SLIIT on how to conduct research by students will be followed.

3.5 Validation Strategy and Performance Metrics

Metric	Target Value	Measurement Method
Message delivery success rate	**>= 90% within range**	Automated test script (100 messages)
Delivery confirmation latency	**<= 2 seconds**	Timestamp delta logging
Operational BLE range (outdoor)	**>= 30 meters**	Field testing at marked distances
AES-128 encryption overhead	**<= 10 ms per message**	Device profiler / timing logs
Group message delivery (5 devices)	**>= 85% success rate**	Multi-device test scenario
App battery consumption per hour	**<= 5% increase vs idle**	Android battery stats API

4. HIGH-LEVEL SYSTEM ARCHITECTURE

The Short-Range Direct Communication Layer is one of the four unified entities in the entire system infrastructure. It acts as the primary component in the entire system infrastructure with regard to the entire process of communication. All the devices using BLE technology are controlled through this component. The additional components, including Decentralised Multi-Hop Routing, Context-Aware Communication Manager, and Delay-Tolerant Message Transport, are connected through a particular API.

4.1 Overall Group System Context

The following illustration depicts the position of the Short-Range Direct Communication Layer in the system framework. The Context-Aware Communication Manager (CACM) interacts with the Short-Range Direct Communication Layer after a target device is detected in BLE proximity. It offers two interface operations: sendDirectMessage(targetDeviceId, encryptedPayload) and broadcastGroupMessage(encryptedPayload), both of which return a status update after successful transmission.

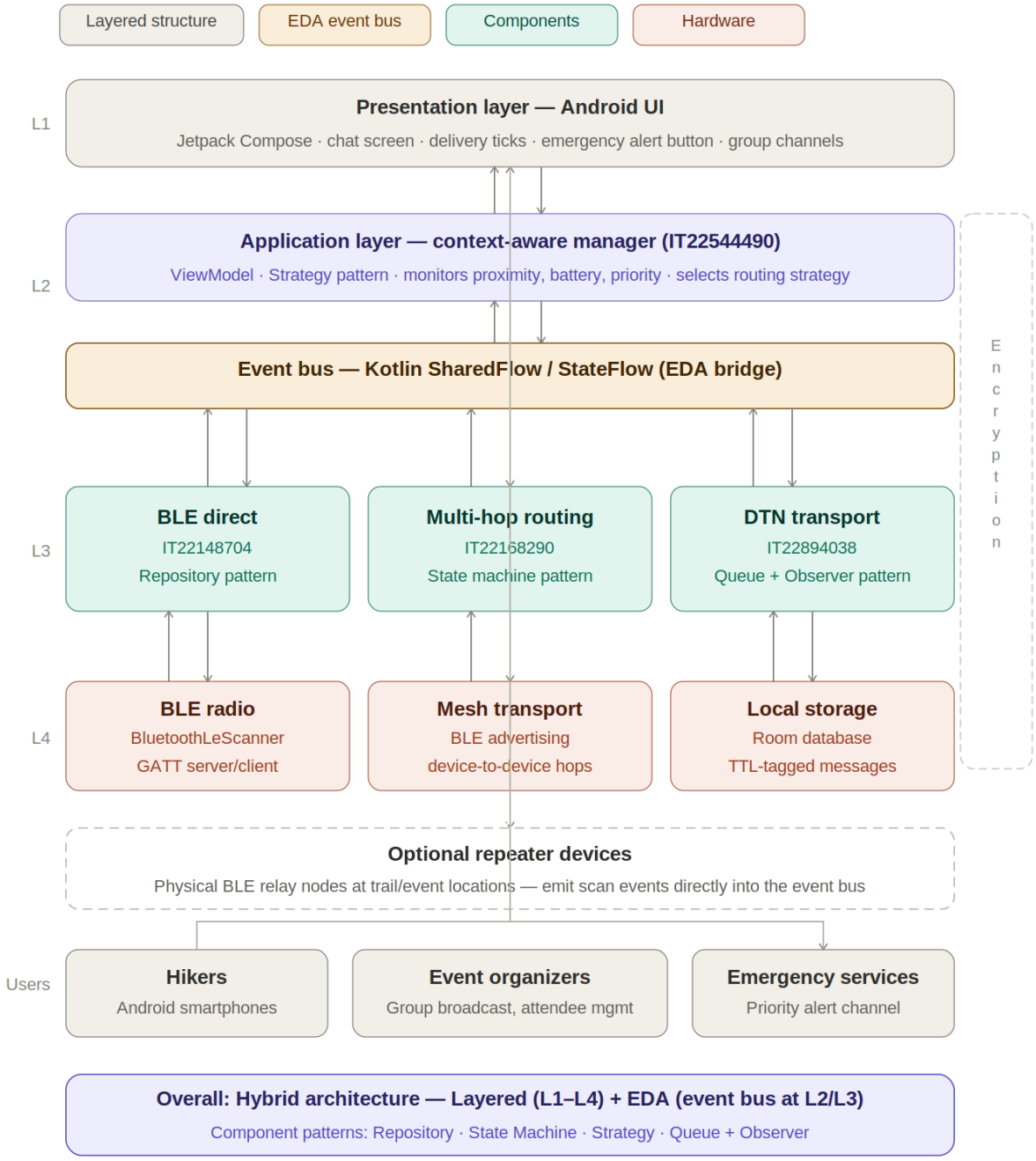


Figure 2*- System Overall platform architecture*

4.2 Component Internal Architecture

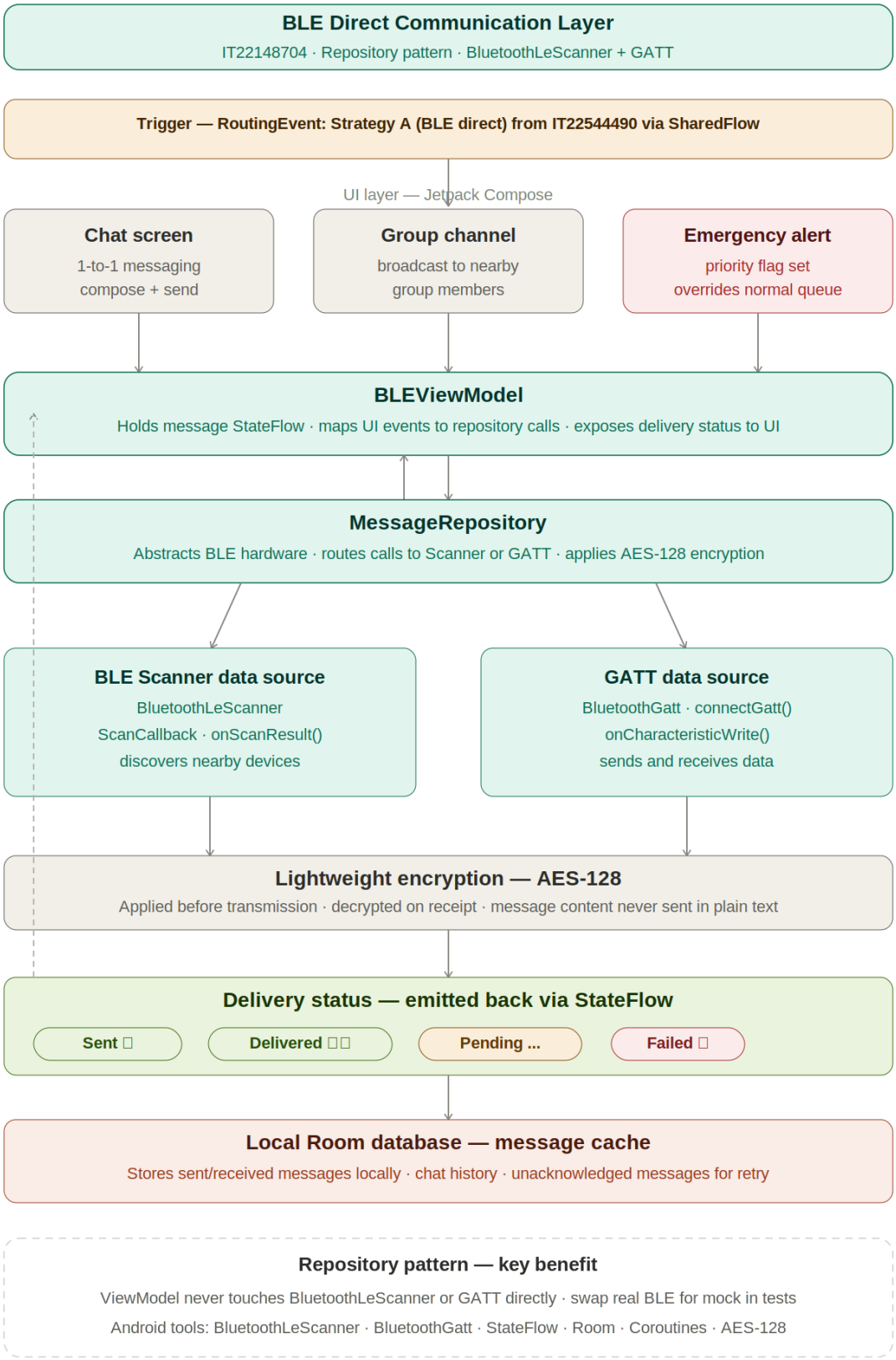


Figure 3 *– Internal architecture*

The Short-Range Direct Communication Layer is internally structured into four sub-modules:

Sub-Module	Responsibilities
BLE Device Scanner	Continuously scans for nearby BLE peripherals advertising the custom Service UUID. Filters discovered devices, maintains a live neighbor list, and triggers UI device discovery notifications. Uses low-latency scan mode when the app is foregrounded.
GATT Communication Manager	Manages BLE connections in both Central (client) and Peripheral (server) roles. Creates and manages the custom GATT service with Message and ACK characteristics. Handles connection lifecycle events (connect, disconnect, MTU negotiation). Implements retry logic for failed transmissions.
Encryption Engine	Performs AES-128 CBC encryption and decryption of all message payloads. Manages session key derivation at connection setup. Generates a new IV per message and prepends it to the ciphertext. Handles key storage in Android Keystore for secure key management.
Delivery Status Manager	Tracks each message-by-Message ID, maps it to a delivery state (Pending, Sent, Delivered, Failed). Starts a 5-second ACK timeout per message. Updates the UI via a state management stream (Riverpod/Provider). Persists delivery state in local SQLite database.

4.3 Data Flow

1. The user composes a message in the Chat Screen (one-to-one or group) and taps Send.
2. The BLEViewModel receives the UI event and calls the MessageRepository with the message content and target device.
3. The Delivery Status Manager generates a unique 4-byte Message ID and sets the initial message status to Pending in the Room database.
4. The MessageRepository passes the message to the Encryption Engine. A 128-bit IV is generated via SecureRandom and the payload is encrypted using AES-128 CBC with the session key established during the GATT connection handshake. The IV is prepended to the encrypted payload.
5. The encrypted packet is assembled into the binary message structure — Message ID, Sender ID, Group Flag, Timestamp, Payload Length, and Encrypted Payload.
6. The GATT Communication Manager (Central role) writes the encrypted packet to the Message Characteristic on the target device's GATT server.
7. The Delivery Status Manager updates the message status from Pending to Sent and starts a 5-second ACK timeout timer.
8. The GATT server on the receiving device processes the incoming characteristic write and calls notifyCharacteristicChanged(), triggering onCharacteristicChanged() on the receiving application.
9. The MessageRepository on the receiving device passes the encrypted payload to its Encryption Engine, which extracts the IV from the head of the payload and decrypts the message content using AES-128 CBC with the shared session key.
10. The decrypted plaintext message is passed to the BLEViewModel on the receiving device and the Chat Screen is updated to display the incoming message.
11. The GATT client on the receiving device sends an ACK packet containing the original Message ID back to the sender's Acknowledgement Characteristic.
12. The sender's GATT server receives the ACK packet on the Acknowledgement Characteristic and passes the Message ID to the Delivery Status Manager.
13. The Delivery Status Manager matches the received Message ID to the pending message, cancels the 5-second timeout, and updates the message status from Sent to Delivered in the Room database.
14. The BLEViewModel observes the StateFlow status change and the Chat Screen updates the delivery indicator from a single tick (Sent) to a double tick (Delivered).

15. If no ACK is received within 5 seconds, the Delivery Status Manager cancels the timeout, updates the message status to Failed in the Room database, and notifies the user via the Chat Screen.
16. The Failed message is retained in the Room database as an undelivered record, available for retry when the target device comes back within BLE range or is forwarded by the DTN component.

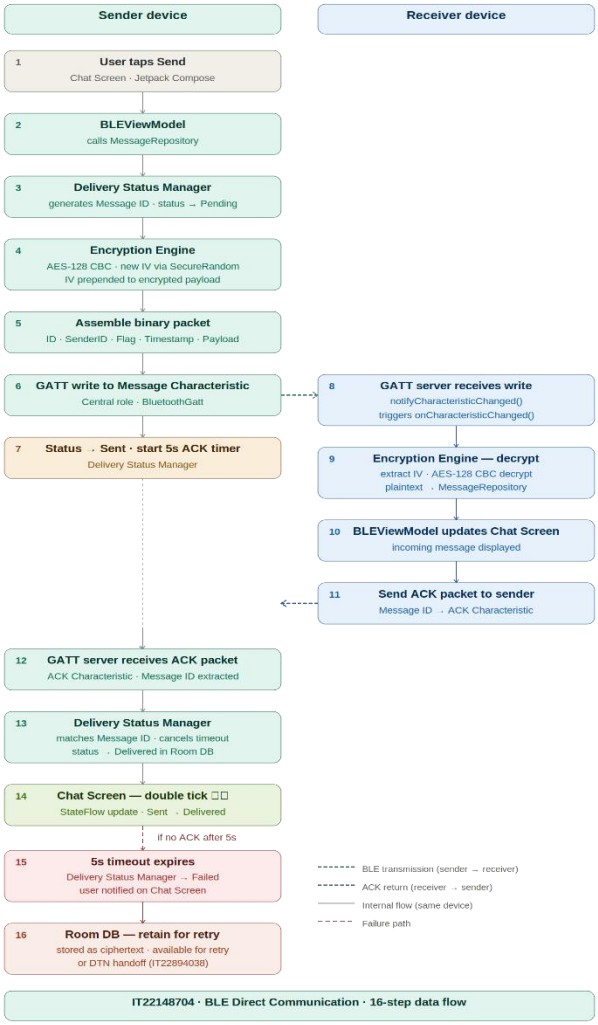


Figure 4 *– Data flow diagram*

4.4 Scalability and Security Considerations

Scalability: The system is currently intended for a cluster size of 2 to 8 devices within the Bluetooth Low Energy range. This is adequate for the intended application (small teams of hikers, staff at events). The maximum number of BLE connections on Android (between 7 and 10 simultaneous connections) is acknowledged as a limitation. Group messages are sent via BLE broadcast advertising (non-connective advertisers), which allows for a lower number of connection overheads for larger groups.

Security: The message content is not vulnerable to eavesdropping due to the implementation of AES-128 encryption. The method for deriving session keys, though simplified for the context, provides a basic level of forward security for the session. Device identifiers are not exposed by the use of hashed MAC addresses. At any point, no message content is sent or stored without being encrypted.

5. USER REQUIREMENTS

5.1 Stakeholder Identification

• Main Users: Hikers and members of trekking teams who have to communicate with each other in areas where there is no cellular connectivity.

• Main Users: Event organizers and members of staff who have to communicate reliably over short distances during events.

• Main Users: Emergency responders who use their smartphones for communication in areas affected by disaster.

• Secondary Users: Developers of the remaining three components who use the API provided in this layer.

5.2 Functional Requirements

ID	Category	Requirement
FR1	Device Discovery	The system shall automatically scan for and display nearby BLE-enabled devices running the application within a 50-metre radius.
FR2	Direct Messaging	The system shall allow a user to send a text message to a specific discovered device without internet connectivity.
FR3	Group Messaging	The system shall allow a user to broadcast a text message to all currently discovered devices simultaneously.

FR4	Delivery Status — Sent	The system shall display a single-tick indicator on a message as soon as the BLE write operation completes successfully.
FR5	Delivery Status — Delivered	The system shall display a double-tick indicator on a message upon receiving an acknowledgement from the recipient device within 5 seconds.
FR6	Delivery Status — Failed	The system shall display a failed indicator and notify the user if no acknowledgement is received within 5 seconds.
FR7	Encryption	The system shall encrypt all message payloads using AES-128 before BLE transmission and decrypt upon receipt.
FR8	Chat History	The system shall store all sent and received messages with their delivery status in a local SQLite database, persisted across app restarts.
FR9	Connection Management	The system shall automatically attempt reconnection if a BLE connection is dropped unexpectedly.
FR10	Internal API	The system shall expose sendDirectMessage() and broadcastGroupMessage() methods for use by the Context-Aware Communication Manager component.

5.3 Non-Functional Requirements

Category	Requirement
Performance	Message delivery confirmation latency shall be <=2 seconds for devices within operational range.
Reliability	Message delivery success rate shall be >=90% for devices within 30 meters in open outdoor environments.
Security	All message content shall be AES-128 encrypted; no plaintext payloads shall be transmitted over BLE.

Usability	A non-technical user (hiker, event staff) shall be able to discover devices and send a message within 60 seconds of launching the app for the first time.
Compatibility	The application shall support Android 8.0 (API level 26) and above, covering the majority of active Android devices.
Power Efficiency	The BLE scanning mode shall consume no more than 5% additional battery per hour compared to idle state.
Portability	The component shall integrate with the other three group components through a defined internal Dart API.

6. COMMERCIALISATION PLAN

6.1 Target Market and Customer Persona

The major market segments would be: (a) Organizations that offer outdoor activities and excursions like hiking, trekking, and adventure travel options; (b) Organizations that manage events, which include organizing outdoor celebrations, sports, and pop-up markets; (c) Organizations that manage emergency operations and non-governmental organizations that manage disaster relief operations and require affordable offline communication options for their members in the field; (d) Military and police training operations in areas that have limited communication.

The customer profile that can be targeted would be that of a small or medium-sized organization in Sri Lanka or Southeast Asia that offers outdoor tours and has 10-20 field guides that they currently manage. These guides currently use expensive walkie-talkie devices that cost between LKR 8,000 and 20,000. This platform has the potential to replace these walkie-talkie devices with either a free or low-cost application on their existing smartphone devices.

6.2 Value Proposition

• No cost for hardware – it can be utilized on any smartphone that has Android 8.0 or a later edition that a user already has.

• Instant delivery acknowledgments – a special feature that is not found in other competing offline messaging services.

• End-to-end encryption – protects important messages during emergency situations.

• No recurring subscription fees; SIM card is not necessary – eliminates recurring charges associated with communicating.

6.3 Revenue Model

• Freemium Mobile Application: A free version available for individual use, allowing for a maximum of 5 devices, and a subscription option available for businesses, allowing for unlimited devices, with a control panel for administrators, for a fee of LKR 1,500/month, i.e., USD 5.

• SDK Licensing: Licensing the BLE communication SDK to various application developers who use offline messaging solutions, for example, navigation applications for hiking, event management applications, etc.

• Enterprise Support Agreements: Yearly agreements for support and customization, targeting businesses responsible for emergency management and event management.

6.4 Cost Estimation

Development costs are minimal for this academic project as all tools are free and open-source.

Post-graduation commercialization costs are estimated as follows:

Item	Cost (LKR)	Notes
Google Play Store Developer Account	8,500 (one-time)	USD ~25 one-time fee
Cloud server (API/dashboard hosting)	5,000/month	DigitalOcean basic droplet
SSL certificate	Free	Let's Encrypt
Marketing (social media)	10,000/month	Targeted campaigns
Android test devices (for development)	60,000 (one-time)	3x budget Android phones

6.5 Competitive Advantage

• The only offline messaging solution that is completely based on a software-only solution, with delivery confirmations comparable to WhatsApp on standard smartphones.

• Created in Sri Lanka, where the local market was recognized and where there is a need for affordable outdoor tourism and areas where disasters often strike.

• A well-documented internal API for integration with the broader group solution, enabling multi-hop routing and DTNs.

• All tools required for development are freely available and open-source, ensuring there are no concerns over intellectual property rights.

6.6 Intellectual Property

The design of the BLE GATT profile used in the message exchange framework and the method of ensuring message delivery can be considered an innovative technology that may be protected under a software patent or registered as a utility model in Sri Lanka. Alternatively, the project may be implemented as an open-source project with the aim of attracting community support, with the potential of earning money through hosting, support, and customizations of the project for businesses.

7. BUDGET AND JUSTIFICATION

Another fundamental guideline of the design of this initiative is the reduction of costs, making it affordable for university students in Sri Lanka. All the selected software applications are free and modifiable. The budget outlined below includes only the necessary hardware and the necessary operating costs that are considered necessary for development and evaluation.

Item	Qty	Cost (LKR)	Justification
Android test smartphone (budget model)	1	15,000–25,000	Required for multi-device BLE testing; uses personal devices + 1 extra
USB cable / OTG adapter	2	500	For USB debugging and ADB testing
Android native	—	Free	Open-source platform

Android native	—	Free	Open-source
pointycastle (encryption)	—	Free	Open-source Dart crypto library
Room base	—	Free	Google standard SQLite abstraction. WAL mode in one line. Compile-time SQL verification catches errors at build time. Flow-based reactive queries. Crash-safe writes.
Android Studio IDE	—	Free	Google's official Android IDE
Git / GitHub	—	Free	GitHub free tier for students
Research paper access (IEEE Xplore)	—	Free (SLIIT library)	SLIIT library provides IEEE Xplore access
Total Estimated Cost		**~16,000–26,000 LKR**	Predominantly hardware only

8. WORK BREAKDOWN STRUCTURE (WBS)

The WBS provided below describes all the tasks related to the Short-Range Direct Communication Layer section, as per various phases. The effort is given in hours, keeping in view a semester of 20 weeks, with approximately 10 to 15 hours per week for this specific component

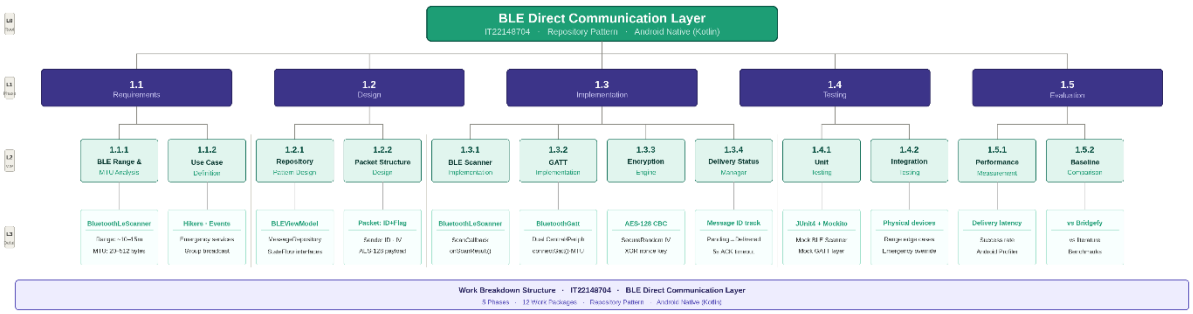


Figure 5 *– Work breakdown Structure (WBS)*

9. GANTT CHART

The Gantt chart below shows the 18-week project timeline (March 2026 – July 2026) for the Short-Range Direct Communication Layer component. Colored blocks indicate active weeks for each task.

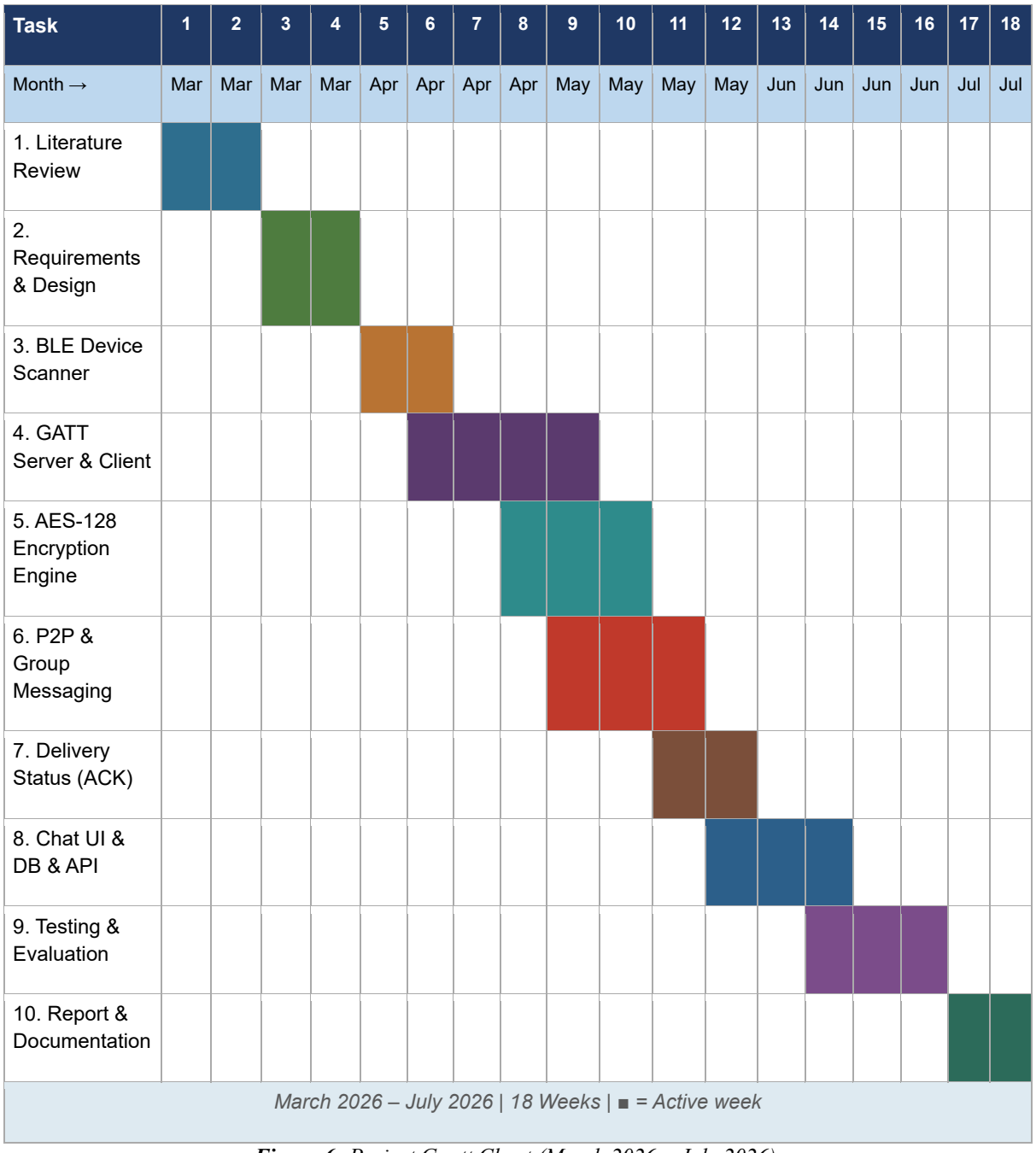


***Figure 6**: Project Gantt Chart (March 2026 – July 2026).*

10. REFERENCES LIST

[1] M. D. Hossain and E. Hossain, "Security and Privacy Threats for Bluetooth Low Energy in IoT and Wearable Devices: A Comprehensive Survey," *IEEE Open Journal of the Communications Society*, vol. 3, pp. 296–341, Feb. 2022, doi: 10.1109/OJCOMS.2022.3149732.

[2] S. M. Gomez, C. Gomez, and J. Paradells, "Bluetooth Low Energy Mesh Networks: A Standards Perspective," *IEEE Wireless Communications*, vol. 27, no. 3, pp. 30–36, Jun. 2020, doi: 10.1109/MWC.001.1900455.

[3] R. Singh, A. N. S. Chauhan, and H. Tewari, "Blockchain-Enabled End-to-End Encryption for Instant Messaging Applications," in *Proc. IEEE 23rd Int. Symp. World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, Belfast, UK, 2022, pp. 501–506, doi: 10.1109/WoWMoM54355.2022.00078.

[4] V. A. Thakor, M. A. Razzaque, and M. R. A. Khandaker, "Lightweight Cryptography Algorithms for Resource-Constrained IoT Devices: A Review, Comparison and Research Opportunities," *IEEE Access*, vol. 9, pp. 28177–28193, 2021, doi: 10.1109/ACCESS.2021.3052867.

[5] A. S. D. Alluhaidan and P. Prabu, "End-to-End Encryption in Resource-Constrained IoT Devices," *IEEE Access*, vol. 11, pp. 70040–70051, 2023, doi: 10.1109/ACCESS.2023.3293382.

[6] J. Vaz, J. C. B. Mattos, and R. I. Soares, "Improving an Ultra Lightweight AES for IoT Applications," in *Proc. IEEE World Forum on Internet of Things (WF-IoT)*, Aveiro, Portugal, Oct. 2023, doi: 10.1109/WF-IoT58464.2023.10539424.

[7] R. Lent, "Adaptive DTN Routing: A Neuromorphic Networking Perspective," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 3, pp. 871–880, Dec. 2020, doi: 10.1109/TCCN.2020.3044369.

[8] W. Wu, J. Guo, Y. Zhou, H. Shen, and L. Liu, "Vehicular Delay Tolerant Network Routing Algorithm Based on Bayesian Network," *IEEE Access*, vol. 8, pp. 18727–18740, Jan. 2020, doi: 10.1109/ACCESS.2020.2968004.

[9] L. Wu, S. Cao, Y. Chen, J. Cui, and Y. Chang, "An Adaptive Multiple Spray-and-Wait Routing Algorithm Based on Social Circles in Delay Tolerant Networks," *Computer Networks*, vol. 189, p. 107901, Apr. 2021, doi: 10.1016/j.comnet.2021.107901.

[10] M. Yousefpour, A. Farahzadi, J. Rezazadeh, and A. Bagheri, "A Survey on Indoor Positioning Systems for IoT-Based Applications," *IEEE Internet of Things Journal*, vol. 9, no. 10, pp. 7680–7699, May 2022, doi: 10.1109/JIOT.2022.3149048.

11. APPENDICES

Appendix A – BLE GATT Profile Specification

Custom GATT service and characteristics used by the BLE Direct Communication Layer . All UUIDs are custom-defined for this platform

Attribute	UUID	Properties	Description
Custom Service	0xFFF0 (custom)	—	Main GATT service for offline BLE messaging. Advertised by all devices running the platform so peers can discover and connect.
Message Characteristic	0xFFF1	WRITE + NOTIFY	Carries encrypted message payloads. The Central (sender) writes the binary packet to this characteristic. The Peripheral (receiver) is notified on each write via onCharacteristicChanged().
ACK Characteristic	0xFFF2	WRITE + NOTIFY	Carries delivery acknowledgements. After successfully receiving and decrypting a message, the receiver writes an ACK packet containing the original Message ID back to this characteristic on the sender's GATT server.
Device Info Characteristic	0xFFF3	READ	Exposes the hashed device UUID used as the Sender ID in the message packet. The UUID is generated on first launch and stored in SharedPreferences — not the Bluetooth MAC address, which is randomised on Android 6.0+.
Key Exchange Characteristic	0xFFF4	WRITE + NOTIFY	Used during session setup to exchange 128-bit nonces for AES-128 session key derivation.

Attribute	UUID	Properties	Description
			Each device writes its random nonce here; the shared session key is computed via XOR of both nonces once the exchange is complete.

Appendix B – Risk Analysis

Risk	Likelihood	Impact	Mitigation
Android BLE API inconsistencies across devices and manufacturers	High	High	Test on multiple Android versions (8.0–14) and device manufacturers (Samsung, Google Pixel, Xiaomi). Use only the native Android BLE API (android.bluetooth.le) — avoid third-party wrappers. Handle connection failure callbacks explicitly in BluetoothGattCallback.
BLE range insufficient for some outdoor and event scenarios	Medium	Medium	Document range limitations clearly in the report. This component handles short-range direct communication only. Longer distances are handled by the multi-hop routing component (IT22168290) and DTN component (IT22894038). Optional repeater devices extend coverage further.
AES-128 encryption overhead impacting message delivery speed	Low	Low	Benchmark AES-128 CBC encryption and decryption time early using Android Profiler. AES-128 on modern ARM CPUs is negligible (<5ms). If overhead is measured above acceptable threshold, reduce IV size or benchmark alternative padding modes.
Integration conflicts with other group	Medium	Medium	Define and agree on the SharedFlow RoutingEvent interface and message packet structure with IT22544490,

Risk	Likelihood	Impact	Mitigation
components via the event bus			IT22168290, and IT22894038 before Week 4 of implementation. Use a shared data model class to avoid type mismatches across components.
Insufficient physical Android test devices available	Medium	High	Coordinate with all four group members to pool personal Android devices for integration testing. A minimum of three devices is required to test multi-hop scenarios. Use Android Emulator with BLE simulation for unit and ViewModel testing where physical devices are unavailable.
XOR nonce session key exchange is cryptographically weak	Low	Medium	Acknowledge this as a known limitation in the report. The XOR nonce approach is accepted given the short-range, offline, low-threat environment and undergraduate project scope. Recommend full ECDH-based key exchange as future work. Document the limitation clearly in the technical limitations section.
Background BLE scanning throttled by Android OS	Medium	Medium	Run the BLE scanner inside a Foreground Service to prevent Android 8.0+ background process restrictions from throttling scan intervals. Display a persistent notification as required by Android policy. Switch to low-power scan mode when the app is in the background and battery is low.